`A11` Atividades

📕 Atividades

A classe `Activity` é um componente crucial de um app para Android, e a maneira como as atividades são lançadas e reunidas é uma parte fundamental do modelo de aplicativo da plataforma. Diferentemente dos paradigmas de programação em que os apps são lançados com um método main(), o sistema Android inicia o código em uma instância Activity invocando métodos de callback que correspondem a estágios específicos do ciclo de vida [1].

A experiência em apps para dispositivos móveis é diferente da versão para computador, porque a interação do usuário com o app nem sempre começa no mesmo lugar. Em vez disso, a jornada do usuário geralmente começa de forma não determinista. Por exemplo, se você abrir um app de e-mails na tela inicial, poderá ver uma lista de e-mails. Por outro lado, se você estiver usando um app de mídia social que inicialize seu app de e-mails, poderá ir diretamente para a tela do app de e-mails para escrever uma mensagem [1].

A classe `Activity` foi desenvolvida para facilitar esse paradigma. Quando um app invoca outro, o app de chamada invoca uma atividade no outro, em vez do app como um todo. Dessa forma, a atividade serve como ponto de entrada para a interação de um app com o usuário. Você implementa uma atividade como uma subclasse da classe `Activity` [1].

Uma atividade fornece a janela na qual o app desenha a própria IU. Essa janela normalmente preenche a tela, mas pode ser menor do que a tela e flutuar sobre outras janelas. Geralmente, uma atividade implementa uma tela em um app. Por exemplo, uma das atividades de um app pode implementar uma tela Preferências, enquanto outra atividade implementa uma tela Selecionar foto [1].

A maioria dos apps contém várias telas, o que significa que elas abrangem várias atividades. Normalmente, uma atividade em um app é especificada como a

atividade principal, que é a primeira tela a ser exibida quando o usuário inicia o app. Cada atividade pode iniciar outra para realizar ações diferentes. Por exemplo, a atividade principal em um app de e-mails simples pode fornecer a tela que mostra uma caixa de entrada de e-mail. A partir daí, a atividade principal pode iniciar outras atividades que fornecem telas para tarefas como escrever e-mails e abrir e-mails individuais [1].

Embora as atividades funcionem juntas para formar uma experiência do usuário coesa em um app, cada uma é vagamente vinculada às outras. Geralmente, há dependências mínimas entre as atividades em um app. Na verdade, com frequência elas iniciam atividades pertencentes a outros apps. Por exemplo, um app de navegação pode iniciar a atividade "Compartilhar" de um app de mídia social [1].

📔 Manifesto

Para que seu app possa usar as atividades, você precisa declará-las junto a alguns dos atributos delas no manifesto [1].

Para declarar sua atividade, abra o arquivo de manifesto e adicione um elemento `<activity>` como filho do elemento `<application>` [1]. Exemplo:

```
<manifest ... >
  <application ... >
      <activity android:name=".MinhaActivity" />
      ...
  </application ... >
  ...
</manifest >
```

O único atributo obrigatório para esse elemento é `android:name`, que especifica o nome da classe da atividade. Você também pode adicionar atributos que definem características de atividade, como etiqueta, ícone ou tema da IU [1].

📗 Ciclo de vida

À medida que o usuário navega no aplicativo, sai dele e retorna a ele, as instâncias Activity no aplicativo transitam entre diferentes estados no ciclo de vida. A classe Activity fornece uma quantidade de callbacks que permite que a atividade saiba sobre a mudança do estado: informa a respeito da criação, interrupção ou retomada de uma atividade ou da destruição do processo em que ela reside por parte do sistema [2].

Dentro dos métodos de callback do ciclo de vida, você pode declarar como a atividade deve se comportar quando o usuário sai e retorna dela. Por exemplo, se estiver construindo um reprodutor de vídeos de transmissão em sequência, você pode pausar o vídeo e encerrar a conexão da rede quando o usuário alternar para outro aplicativo. Quando o usuário retornar, será possível reconectar a rede e permitir que ele reinicie o vídeo de onde parou. Ou seja, cada callback permite que você realize o trabalho específico adequado a determinada mudança de estado [2].

Fazer o trabalho certo no momento apropriado e gerenciar as transições da maneira correta faz com que seu aplicativo seja mais robusto e tenha melhor desempenho. Por exemplo, uma boa implementação dos callbacks de ciclo de vida pode ajudar a garantir que seu aplicativo evite os problemas a seguir [2]:

- Falhas se o usuário receber uma chamada telefônica ou mudar para outro aplicativo enquanto estiver usando seu aplicativo.
- Consumo de recursos importantes do sistema quando o usuário não estiver usando ativamente o aplicativo.
- Perda do progresso do usuário se ele sair do aplicativo e retornar mais tarde.

Conceitos do ciclo de vida da atividade

Para navegar entre as fases do ciclo de vida da atividade, a classe “Activity” fornece um conjunto principal de seis callbacks: onCreate(), onStart(), onResume(), onPause(), onStop() e onDestroy(). Conforme a atividade entra em um

novo estado, o sistema invoca cada um desses callbacks. A Fig. 1 demonstra a representação visual desse paradigma [2].

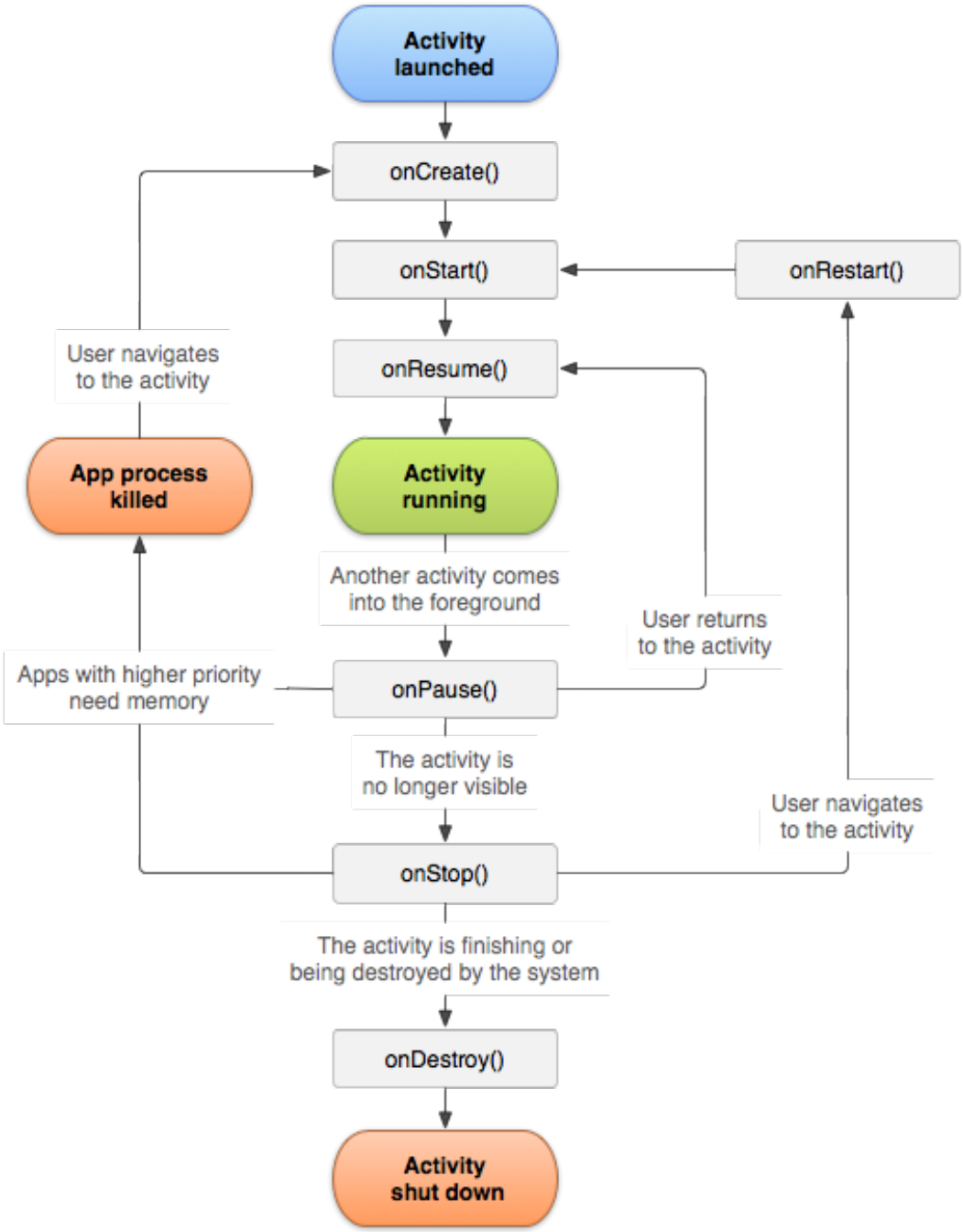

Figura 1: Ilustração simplificada do ciclo de vida da atividade.

Ao longo da vida útil de uma atividade, ela passa por vários estados. Uma série de callbacks são usados para lidar com transições entre estados. As seções a seguir apresentam esses callbacks [1].

`onCreate()`

Você precisa implementar esse callback, que é acionado quando o sistema cria sua atividade. A implementação inicializará os componentes essenciais da atividade. Por exemplo, aqui, o app criará visualizações e vinculará dados a listas. Mais importante, é nesse local que você precisa chamar `setContentView()` para definir o layout da interface do usuário da atividade [1].

`onStart()`

Quando `onCreate()` sai, a atividade entra no estado "Iniciado" e se torna visível para o usuário. Esse callback contém o que equivale aos preparativos finais da atividade para ir para o primeiro plano e se tornar interativa [1].

`onResume()`

O sistema invoca esse callback imediatamente antes de a atividade começar a interagir com o usuário. Neste ponto, a atividade fica na parte superior da pilha de atividades e captura toda a entrada do usuário [1].

`onPause()`

O sistema chama `onPause()` quando a atividade perde o foco e entra em um estado "Pausado". Esse estado ocorre quando, por exemplo, o usuário toca no botão "Voltar" ou "Recentes". Quando o sistema chama `onPause()` para sua atividade, isso significa, que ela ainda está parcialmente visível. No entanto, na maioria das vezes, é uma indicação de que o usuário está deixando a atividade e que logo ela entrará no estado "Interrompido" ou "Retomado" [1].

Uma atividade no estado "Pausado" pode continuar atualizando a IU se o usuário estiver esperando por isso. Exemplos de uma dessas atividades incluem a exibição da tela de um mapa de navegação ou de um player de mídia sendo reproduzido. Mesmo que essas atividades percam o foco, o usuário espera que a IU continue sendo atualizada [1].

> Não use `onPause()` para salvar dados do app ou do usuário, fazer chama-

> das de rede ou executar transações de banco de dados [1].

`onStop()`

O sistema chama `onStop()` quando a atividade não está mais visível para o usuário. Isso pode acontecer porque a atividade está sendo destruída, uma nova atividade está sendo iniciada ou uma atividade existente está entrando em um estado "Retomado" e está cobrindo a atividade interrompida. Em todos esses casos, a atividade interrompida não fica mais visível [1].

`onRestart()`

O sistema invoca esse callback quando uma atividade no estado "Interrompido" está prestes a ser reiniciada. `onRestart()` restaura o estado da atividade a partir do momento em que ela foi interrompida [1].

O sistema invoca esse callback antes de uma atividade ser destruída. Esse é o último callback que a atividade recebe. `onDestroy()` normalmente é implementado para garantir que todos os recursos de uma atividade sejam liberados quando ela (ou o processo que a contém) for destruída [1].

📘 Ciclo de vida

A janela Logcat no Android Studio exibe mensagens do sistema, como quando ocorre uma coleta de lixo, e mensagens que você adicionou ao seu app com a classe Log. Ela exibe mensagens em tempo real e mantém um histórico para que você possa ver mensagens mais antigas [3].

A classe Log permite que você crie mensagens de registro que aparecem no Logcat. Geralmente, os seguintes métodos de registro são usados, listados em ordem decrescente de prioridade (ou do menos para o mais detalhado) [3]:

```
Log.e(String, String) // (erro)
Log.w(String, String) // (aviso)
Log.i(String, String) // (informações)
```

```
Log.d(String, String) // (depuração)
Log.v(String, String) // (detalhado)
```

Nunca compile registros detalhados no seu app, exceto durante o desenvolvimento. Registros de depuração são compilados, mas eliminados no tempo de execução, enquanto os registros de erro, aviso e informações sempre são mantidos [3].

Para cada método de registro, o primeiro parâmetro precisa ser uma tag exclusiva e o segundo, uma mensagem. A tag de uma mensagem de registro do sistema é uma string curta que indica o componente do sistema a partir do qual a mensagem é originada. A tag pode ser qualquer string que você achar útil, como o nome da classe atual [3].

Uma boa prática geral é declarar uma constante `TAG` na classe para usar no primeiro parâmetro. Por exemplo, você pode criar uma mensagem de informação de registro da seguinte forma [3]:

```
private const val TAG = "MyActivity"
...
Log.i(TAG, "MyClass.getView() — get item number $position")
```

👨🏼‍💻 Atividade prática

1. Crie um projeto Android chamado `Aula11`.
2. Na classe `MainActivity`, sobrescreva todos os métodos de estado.
3. Em cada método, imprima log o nome dele.
4. Execute a aplicação e observe as mensagens no logcat.

📚 Referência bibliográficas

[1] Atividades. **Android Developers**. Acessado em 14/01/2021.

[2] Ciclo de vida. **Android Developers**. Acessado em 14/01/2021.

[3] Logcat. **Android Developers**. Acessado em 14/01/2021.